AquaCanvas: Real-Time Watercolor Simulation

CS 184/284A Final Project Proposal, Spring 2026

Team Dubuworld

Daeyoung Kim, Jaegyum Kim, Brian Sui, Shenrui Wu

https://dubu911.github.io/184_final_project_team_dubuworld/proposal.html

Summary

We want to build a digital painting tool that simulates how watercolor actually works. Water flows on a canvas, carries pigment around, and dries over time. Instead of faking the look with textures we plan to run a real fluid simulation so users can paint and watch the water behave naturally. We are building this in the Godot game engine and using compute shaders for the physics.

Problem Description

Watercolor is one of the hardest traditional media to control. Water pools up, pigment bleeds into wet areas, colors mix in ways you cannot fully predict, and you only see the final result after everything dries. That unpredictability is also what makes it beautiful.

Most digital painting tools today like Photoshop, Procreate, and Clip Studio are designed around opaque media like oil paint or acrylic. Their brush engines work well for those but watercolor is a different problem. Simulating water flow in real time is computationally expensive so existing watercolor features in these tools are mostly static. They try to approximate the final dried look without actually simulating the fluid behavior underneath. You do not get to watch the water spread or see two wet strokes bleed into each other like they would on real paper.

We want to try a different approach. We want to simulate the fluid dynamics directly and let the visual effects come from the physics. One of our team members (Daeyoung) has a background in traditional painting including watercolor so we have a good sense of what the simulation should look and feel like from an artist's perspective. That experience will help us decide which physical effects matter most and how the brush should respond.

The main challenges we see are:

- **Performance.** Running a fluid simulation per pixel across the whole canvas every frame is expensive. A straightforward CPU loop is too slow for interactive use so we need to move the computation to GPU compute shaders.
- **Coupled physics.** Water flow, pigment transport, diffusion, evaporation, and deposition are all connected. Pigment moves with the water, diffuses when wet, and sticks to the paper as the water dries. Getting these to work together without blowing up or looking wrong is the main technical challenge.
- **Looking right.** Even if the physics is correct the result needs to actually look like watercolor. Things like wet-on-wet bleeding, darker edges where pigment accumulates, smooth gradients. As someone who has painted with real watercolor Daeyoung will be checking whether the output feels authentic or just looks like a physics demo.
- **Input latency.** The tool has to feel responsive when you paint. If there is a noticeable delay between the brush stroke and the water appearing it is not usable as a painting tool.

Our plan is to model the canvas as a grid where each cell tracks water height, mobile pigment (floating in water), static pigment (dried onto paper), and paper absorbency. We will write the simulation stages as GLSL compute shaders and dispatch them through Godot's `RenderingDevice` API. We will start with a CPU version first to get the physics right then port it to the GPU to scale up.

Goals and Deliverables

What We Plan to Deliver (Baseline)

This is what we think is realistic and what we need for a successful project.

- **A working watercolor simulation** with these physics stages.
  1. Water displacement. Gravity and surface tension moving water around.

2. Inflow. Redistributing water between neighboring cells.
3. Pigment diffusion. Pigment spreading through wet areas.
4. Evaporation. Water drying over time.
5. Deposition. Pigment sticking to the paper as water evaporates.

We will first get these working on the CPU at a small canvas size to make sure the physics is correct then port them to GPU compute shaders and scale the canvas up.

- **A watercolor brush** that deposits water and pigment onto the canvas. We would like to support pressure sensitivity with a Wacom tablet but will fall back to mouse input if that is too hard to get working.
- **Minimal UI** with brush size control, color picker, and a way to toggle layer visibility for debugging.
- **CPU vs. GPU comparison.** We will keep our CPU prototype around and measure the FPS difference to show the benefit of GPU acceleration.
- **Demo images and a painting video.** We will produce sample paintings showing recognizable watercolor effects like bleeding, edge darkening, and dripping. We will also record a video of someone actually painting with the tool to show the interactive experience.

What We Hope to Deliver (Aspirational)

If things go well and we have time left.

- **Extra brush types.** A pencil for sketching, an eraser, and a removing brush that lifts pigment off the canvas.
- **Physics settings panel.** Sliders to tweak surface tension, evaporation rate, etc. while painting.
- **Momentum tracking.** An inertia map to make water flow look smoother and reduce oscillation.
- **PNG export** so you can save your painting.

How We Will Measure Success

- **Performance.** FPS comparison between CPU and GPU versions. We want the GPU version to run at interactive frame rates (at least 30 FPS).
- **Visual quality.** Sample paintings compared with real watercolor references. We will look for characteristic effects like bleeding, edge darkening, and gravity-driven flow.

- **Responsiveness.** The brush should feel responsive enough to actually paint with. No obvious lag.

Questions We Want to Answer

- Can we run a physically based watercolor simulation fast enough for real time painting using GPU compute shaders?
- How much faster is the GPU version compared to the CPU version?
- What is the minimum set of physics (surface tension, diffusion, evaporation, deposition) we need to get something that actually looks like watercolor?

Schedule

Week	Dates	Tasks
1	Apr 7 to Apr 13	• Set up Godot project and get a canvas rendering on screen • Build CPU prototype with water displacement and evaporation • Basic brush that deposits water and pigment
2	Apr 14 to Apr 20	• Finish CPU physics (diffusion, inflow, deposition) • Start GPU port with compute shaders for the heaviest stages • Get tablet pressure input working
3	Apr 21 to Apr 27	• Finish GPU port and validate against CPU reference • Performance benchmarks and UI polish • **Milestone checkpoint**
4	Apr 28 to May 5	• Stretch goals if time allows

		• Record painting demo video and create sample images • **Final presentation on Tuesday, May 5**

Resources

References

- Curtis, C. J., Anderson, S. E., Seims, J. E., Fleischer, K. W., and Salesin, D. H. *Computer-Generated Watercolor*. SIGGRAPH 1997.
- Small, D. *Modeling Watercolor by Simulating Diffusion, Pigment, and Paper Fibers*.
- Chu, N. and Tai, C.-L. *MoXi: Real-Time Ink Dispersion in Absorbent Paper*. SIGGRAPH 2005.
- Van Laerhoven, T. and Van Reeth, F. *Real-Time Simulation of Watery Paint*. Computer Animation and Virtual Worlds, 2005.
- Xu, S., Tan, H., Jiao, X., Lau, F. C. M., and Pan, Y. *A Generic Pigment Model for Digital Painting*.
- *Real-time Watercolor Painting on a Distributed Paper Model*.
- *Real-time Watercolor Simulation with Fluid Vorticity within Brush Stroke*.

Software Platform

- **Engine.** Godot Engine 4.x (open source game engine with a built in `RenderingDevice` API for compute shaders)
- **Shading language.** GLSL 450 (Vulkan compute shaders)
- **Scripting.** GDScript
- **Input.** Wacom tablet for pressure sensitive brush input

Hardware

- We are developing on an i5 with integrated GPU running Windows 11. This is intentionally modest hardware. If we can get interactive frame rates here it should run well on anything with a discrete GPU too.

Starting Point

We are building this from scratch. Godot gives us the rendering infrastructure and compute shader dispatch but all the simulation logic, brush systems, and UI will be our own code. The papers listed above are our main theoretical references for the physics model.